PAI-4 DevSecOps — Reparto de roles y paso a paso (ST20)

Nov 13, 2026 · @Fran

Reparto

Tres roles con carga parecida y una frontera clara: **A** monta la plataforma por la que pasa todo, **B** integra los análisis de la etapa de build y **C** integra el DAST, los gates bloqueantes y la gestión de hallazgos. El diseño completo está en el documento de planteamiento; aquí va quién hace qué y en qué orden.

	Rol A · Plataforma	Rol B · Análisis en build	Rol C · DAST, gates y gestión
En qué consiste	Que el pipeline exista y funcione: repo, app, imágenes, Kubernetes, DefectDojo accesible y la subida automática de informes	Que el código, las dependencias, la infraestructura y la imagen se analicen en cada push y sus hallazgos lleguen a DefectDojo	Que la app en ejecución se ataque, que el pipeline se pare cuando debe y que los hallazgos se gestionen y se puedan demostrar
Herramientas	GitHub Actions, Docker, kind, DefectDojo, Cloudflare Tunnel	Dependency-Check, Semgrep, Checkov, Trivy	OWASP ZAP, `quality_gate.py`, DefectDojo (triage)
Jobs de `devsecops.yml`	`build`, `deploy` y la estructura general	`sca`, `sast`, `iac`, `container`	`dast`, `security-gate`, `notify`
Ficheros que entrega	`Dockerfile` (revisado), `k8s/*`, `scripts/dd-upload.sh`, `defectdojo/README.md`, `.github/dependabot.yml`	`security/dependency-check-suppressions.xml`, `.semgrepignore`, `.checkov.yaml`, `.trivyignore`	`scripts/quality_gate.py`, `security/zap/rules.tsv`, carpeta `evidencias/`
Gates	—	G1, G2, G3, G4 (aviso)	G5, G6 (bloqueantes)
Diagramas	1.a	Revisa 1.a	1.b
Informe	Apartados 1 y 3 (diseño del pipeline)	Apartados 2, 4 (G1–G4) y 5 (sus 4 herramientas)	Apartados 3 (1.b), 4 (G5–G6), 5 (ZAP), 6, 7 y anexos

Dependencias entre roles: B y C no pueden subir nada a DefectDojo hasta que A tenga DefectDojo accesible y `dd-upload.sh` probado, y C necesita el artefacto `image.tar` del job `build`. Por eso A va primero y lo más urgente es que tenga ambas cosas antes del seguimiento del 17/18. Mientras tanto, B y C prueban sus herramientas en local con Docker sobre el código de Juice Shop.

Contrato común para no pisarse:

- Cada rol trabaja en su rama (`rol-a/...`, `rol-b/...`, `rol-c/...`) y entra en `main` por pull request revisado por otro rol.
- Cada job de seguridad sigue el mismo patrón: escanear sin fallar → `dd-upload.sh` con `if: always()` → subir el informe como artefacto `report-<job>` → gate con `quality_gate.py`.
- Nombres de los tests en DefectDojo fijos: `SCA - Dependency-Check`, `SAST - Semgrep`, `IaC - Checkov`, `Container - Trivy`, `DAST - ZAP`.
- Nada de credenciales en el repo; las pide el que las necesite y A las da de alta como secretos.

Rol A · Plataforma

Rol A deja listo el terreno: repositorio con Juice Shop, pipeline que construye y despliega, manifiestos de Kubernetes y DefectDojo recibiendo informes por API. Es el camino crítico: B y C dependen de él para integrar.

En qué consiste: es el papel de quien administra la plataforma de CI/CD en una empresa DevSecOps. No busca vulnerabilidades, pero sin él no hay dónde ejecutarlas ni dónde gestionarlas. También representa el escenario del cliente (contenedores + Kubernetes) y dibuja el diagrama del pipeline.

Fase 1 · Repositorio y aplicación (antes del 17 nov)

1. Crear el repo público `ST20-PAI4-DevSecOps` y dar acceso de escritura a los otros dos.
2. Traer Juice Shop en una versión fijada:

```
git clone --depth 1 --branch <tag-estable> https://github.com/juice-shop/juice-shop.git /tmp/js
rsync -a --exclude .git --exclude .github /tmp/js/ ./
git add . && git commit -m "Juice Shop <tag-estable> como aplicación de prueba"
```

3. Comprobar que construye y arranca en local: `docker build -t juice-shop-st20 .` y `docker run --rm -p 3000:3000 juice-shop-st20`, abrir `http://localhost:3000`. Anotar cuánto tarda el build (la primera vez son varios minutos por `npm install` y Angular).
4. Settings → Actions → General: *Workflow permissions* en solo lectura; los jobs que necesiten más lo piden con `permissions:`.
5. Añadir enlace al repo en `Frarodmar19/SSII`, carpeta `PAI4/README.md`.

Fase 2 · DefectDojo accesible (antes del 17 nov)

1. En la máquina que vaya a quedarse encendida esas semanas (≥ 4 GB de RAM libres):

   ```
   git clone https://github.com/DefectDojo/django-DefectDojo.git
   cd django-DefectDojo
   docker compose up -d
   docker compose logs initializer | grep "Admin password:"
   ```

   Si el README de la versión clonada pide `docker compose build` antes, hacerlo. Interfaz en `http://localhost:8080`.
2. Entrar como `admin`, cambiar la contraseña y crear el usuario `ci-bot` (rol Writer). Copiar su API v2 key.
3. System Settings: activar *Deduplicate findings* y *False positive history*. Crear el Product Type `SSII`.
4. Exponerlo: `cloudflared tunnel --url http://localhost:8080` y apuntar la URL `https://…trycloudflare.com`. Si la API responde 400 por host no permitido, revisar `DD_ALLOWED_HOSTS` en la configuración de Compose.
5. Dar de alta en GitHub los secretos `DD_URL`, `DD_API_KEY` y `NVD_API_KEY` (este último lo solicita B en nvd.nist.gov). Opcional: `DISCORD_WEBHOOK` para C.
6. Escribir `defectdojo/README.md`: cómo se levanta, cómo se expone, cómo se rota la API key. Sin ninguna credencial.

Fase 3 · Script de subida y esqueleto del workflow (antes del 17 nov)

1. Crear `scripts/dd-upload.sh` tal como está en el documento de planteamiento y darle permisos (`git update-index --chmod=+x scripts/dd-upload.sh`).
2. Probarlo en local con un informe cualquiera antes de meterlo en el pipeline:

   ```
   export DD_URL=https://…trycloudflare.com DD_API_KEY=… DD_PRODUCT="Juice
   Shop ST20" DD_ENGAGEMENT="CI/CD GitHub Actions"
   export GITHUB_RUN_ID=local GITHUB_SHA=$(git rev-parse HEAD)
   GITHUB_REF_NAME=main
   scripts/dd-upload.sh "Trivy Scan" trivy.json "Container - Trivy"
   ```

   Comprobar en DefectDojo que se creó producto, engagement y test, y que repetir el comando no duplica hallazgos.
3. Subir `devsecops.yml` con los 8 jobs del esqueleto: `build` completo y el resto vacíos (un `echo`) para que B y C solo rellenen el suyo.
4. Completar `build`: `docker build`, `docker save` y `upload-artifact` con nombre `image`. Activar caché de capas si el build pasa de 10 minutos (`docker/build-push-action` con `cache-from: type=gha`).

Fase 4 · Kubernetes y despliegue (18–24 nov)

1. Escribir los manifiestos en `k8s/` **sin endurecer a propósito** (sin `securityContext`, sin límites de recursos, sin probes), reproduciendo el punto de partida del cliente:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: juice-shop
  namespace: juice-shop
spec:
  replicas: 1
  selector:
    matchLabels:
      app: juice-shop
  template:
    metadata:
      labels:
        app: juice-shop
    spec:
      containers:
        - name: juice-shop
          image: juice-shop-st20:latest
          ports:
            - containerPort: 3000
```

   Más `namespace.yaml` y un `service.yaml` tipo `NodePort` al puerto 3000.
2. Completar `deploy` (`needs: [dast, security-gate]`, `permissions: packages: write`):
   a. Descargar y cargar `image.tar`.
   b. `docker login ghcr.io` con `GITHUB_TOKEN`, etiquetar como `ghcr.io/<owner-en-minúsculas>/juice-shop-st20:<sha>` y hacer push. GHCR exige el nombre en minúsculas.
   c. `helm/kind-action@v1` con `cluster_name: st20`, luego `kind load docker-image ... --name st20`.
   d. `kubectl apply -f k8s/`, `kubectl -n juice-shop set image deployment/juice-shop juice-shop=<imagen:sha>`, `kubectl -n juice-shop rollout status deployment/juice-shop --timeout=180s`.
   e. Smoke test: `kubectl port-forward` en segundo plano y `curl -sf http://localhost:3000`.
3. Añadir `.github/dependabot.yml` (ecosistemas `npm`, `docker` y `github-actions`, semanal) como control SCA continuo fuera del pipeline.

4. Proteger `main`: *Require status checks* con `security-gate` y *Require a pull request before merging*.

Fase 5 · Diagrama e informe (25 nov – 1 dic)

1. Redibujar el diagrama 1.a en draw.io a partir del del planteamiento y comprobar job por job que coincide con `devsecops.yml`. Cualquier diferencia: se cambia el diagrama y se anota el motivo.
2. Escribir el apartado 1 (contexto, respuesta corta a la pregunta del cliente, enlace al repo) y el apartado 3 (diseño del pipeline, cambios respecto al diseño inicial).
3. Evidencias propias: captura del grafo de jobs de un run, captura de los nombres de los secretos, captura del despliegue en `kind` del run de `demo/gate`.

Checklist Rol A:

- [ ] Repo con Juice Shop fijado y sin workflows originales
- [ ] DefectDojo levantado, `ci-bot` creado, túnel funcionando
- [ ] Secretos dados de alta (`DD_URL`, `DD_API_KEY`, `NVD_API_KEY`)
- [ ] `dd-upload.sh` probado: crea contexto y no duplica al repetir
- [ ] Esqueleto de `devsecops.yml` en `main` con `build` funcionando
- [ ] `k8s/` y job `deploy` desplegando en `kind`
- [ ] Dependabot y protección de `main`
- [ ] Diagrama 1.a final y apartados 1 y 3

Rol B · Análisis en build (SCA, SAST, IaC y contenedor)

Rol B integra las cuatro herramientas que analizan sin ejecutar la aplicación: Dependency-Check, Semgrep, Checkov y Trivy. Cada una en su job, en modo aviso (G1–G4), con su informe en DefectDojo y al menos una vulnerabilidad real detectada y explicada.

En qué consiste: es el *shift left* del enunciado. Cubre tres de los cuatro orígenes de vulnerabilidades de la presentación (código propio, dependencias de terceros y configuración) y la cadena de suministro de la imagen. También es quien justifica en el informe por qué se eligieron estas herramientas para una app Node.js/TypeScript.

Fase 1 · Probar cada herramienta en local (antes del 17 nov)

Sobre una copia local del repo de Juice Shop, sin esperar al pipeline. Guardar los informes: sirven para que A pruebe `dd-upload.sh` y para saber qué esperar.

1. **NVD API key:** solicitarla en nvd.nist.gov (llega por correo) y pasársela a A para el secreto `NVD_API_KEY`.

2. **Dependency-Check:**

```
test -f package-lock.json || npm install --package-lock-only --ignore-scripts
docker run --rm -v "$PWD:/src" -v dc-data:/usr/share/dependency-check/data owasp/dependency-check \
  --scan /src --exclude "**/node_modules/**" --format XML --format HTML --out /src/reports \
  --nvdApiKey "$NVD_API_KEY" --project juice-shop
```

   La primera ejecución descarga la base NVD y tarda bastante; el volumen `dc-data` la reutiliza. Anotar las dependencias con CVSS ≥ 7.

3. **Semgrep:**

```
docker run --rm -v "$PWD:/src" semgrep/semgrep semgrep scan \
  --config p/owasp-top-ten --config p/nodejsscan --config p/secrets \
  --json --output /src/semgrep.json /src
```

   Buscar en el resultado inyecciones SQL por concatenación en `routes/` y claves embebidas en `lib/`, que son las clásicas de Juice Shop. Confirmar fichero y línea antes de citarlas en el informe.

4. **Checkov** (cuando A tenga `k8s/`; mientras, solo el `Dockerfile`):

```
docker run --rm -v "$PWD:/code" bridgecrew/checkov -d /code --framework dockerfile kubernetes -o json > checkov.json
```

   Esperable: checks `CKV_K8S_*` por falta de `securityContext`, límites y probes, y `CKV_DOCKER_*` en el Dockerfile.

5. **Trivy:**

```
docker build -t juice-shop-st20 .
docker run --rm -v /var/run/docker.sock:/var/run/docker.sock aquasec/trivy image --format json -o trivy.json juice-shop-st20
```

   Separar lo que viene de la imagen base de lo que viene de los paquetes npm empaquetados: es el argumento de por qué Trivy complementa a Dependency-Check.

Fase 2 · Integrar los 4 jobs (18–24 nov)

En la rama `rol-b/analisis-build`, rellenando los jobs vacíos del esqueleto con el patrón común (escanear sin fallar → `dd-upload.sh` → artefacto `report-<job>` → gate en aviso):

Job	Acción / imagen	Informe	`scan_type` para DefectDojo	Gate (aviso)
`sca`	`dependency-check/Dependency-Check_Action`	`reports/dependency-check-report.xml`	`Dependency Check Scan`	G1: CVSS ⩾ 7.0
`sast`	contenedor `semgrep/semgrep`	`semgrep.json`	`Semgrep JSON Report`	G2: ⩾ 1 `ERROR`
`iac`	`bridgecrewio/checkov-action` con `soft_fail: true`	`reports/results_json.json`	`Checkov Scan`	G3: ⩾ 1 check fallido
`container`	`aquasecurity/trivy-action` con `input: image.tar` y `exit-code: 0`	`trivy.json`	`Trivy Scan`	G4: ⩾ 1 `CRITICAL` con parche

Puntos a vigilar:

1. El paso del gate lleva `continue-on-error: true` y el de subida `if: always()`. Comprobar en Actions que el job sale en verde con anotación de aviso, no en rojo.
2. Verificar el nombre real del fichero que genera cada acción (sobre todo Checkov) y ajustar la ruta en la subida.
3. Si DefectDojo rechaza el JSON de Checkov por traer varios frameworks en una lista, ejecutar Checkov una vez por framework y subir dos informes al mismo test.
4. Plan B si Dependency-Check es demasiado lento incluso con la key: cachear su directorio de datos con `actions/cache` o sustituirlo por `trivy fs` como SCA (justificándolo en el informe).
5. Comprobar en DefectDojo que el mismo CVE detectado por Dependency-Check y por Trivy aparece marcado como duplicado. Es una evidencia del objetivo 5.

Fase 3 · Parsers del gate y ficheros de supresión (24 nov)

1. Añadir a `scripts/quality_gate.py` (lo estructura C) las cuatro funciones que cuentan hallazgos sobre el umbral: máximo `baseScore` CVSS v3 por dependencia en el XML de Dependency-Check, `extra.severity` en `results` de Semgrep, `failed_checks` en Checkov y `Severity` + `FixedVersion` en `Results[].Vulnerabilities` de Trivy.
2. Crear los ficheros de supresión vacíos pero válidos para que el pipeline no falle por no encontrarlos: `security/dependency-check-suppressions.xml`, `.semgrepignore` (excluye `test/` y `*.spec.ts`), `security/.checkov.yaml` y `security/.trivyignore`.

3. Con C, preparar la rama `demo/gate`: añadir a `.trivyignore` y a las supresiones de Dependency-Check los CVE que disparan G6, cada uno con motivo y fecha de revisión.

Fase 4 · Informe (25 nov – 1 dic)

1. Apartado 2: tabla de herramientas y por qué cada una encaja con Node.js/TypeScript + Docker + Kubernetes; por qué no IAST (comercial) ni SonarQube (requiere servidor).
2. Apartado 4, G1–G4: cómo se integró cada herramienta, umbral, por qué en modo aviso.
3. Apartado 5, sus cuatro herramientas: número de hallazgos por severidad y uno o dos ejemplos concretos de cada una (CVE y paquete, regla y fichero:línea, check de Checkov y recurso), con cómo se corregiría.
4. Evidencias propias: resumen de cada job con su tabla de gate y los cuatro tests en DefectDojo.

Checklist Rol B:

- [ ] NVD API key pedida y entregada a A
- [ ] Las 4 herramientas probadas en local con informe guardado
- [ ] Jobs `sca`, `sast`, `iac`, `container` integrados y subiendo a DefectDojo
- [ ] G1–G4 en modo aviso, jobs en verde con anotación
- [ ] Al menos 1 hallazgo real por herramienta identificado y explicado
- [ ] Duplicado Dependency-Check / Trivy visto en DefectDojo
- [ ] Parsers añadidos a `quality_gate.py` y ficheros de supresión creados
- [ ] Apartados 2, 4 (G1–G4) y 5 (sus herramientas)

Rol C · DAST, quality gates y gestión de vulnerabilidades

Rol C integra ZAP contra la app en ejecución, escribe el motor de quality gates, configura los dos gates que bloquean de verdad (G5 y G6) y lleva el triage en DefectDojo, las evidencias y el cierre del informe.

En qué consiste: es quien decide cuándo el pipeline se para y qué pasa con cada hallazgo después. Cubre el cuarto origen de vulnerabilidades (la aplicación en ejecución), los objetivos 4 y 5 y el diagrama 1.b. Al final responde a la pregunta del cliente con los resultados delante.

Fase 1 · ZAP en local y diagrama 1.b (antes del 17 nov)

1. Arrancar Juice Shop (`docker run -d --name js -p 3000:3000 juice-shop-st20`, o la imagen oficial mientras A termina) y lanzar ZAP contra ella:

   ```
   docker run --rm --network host -v "$PWD:/zap/wrk:rw"
   ghcr.io/zaproxy/zaproxy:stable \
     zap-full-scan.py -t http://localhost:3000 -j -a \
     -J report_json.json -x report_xml.xml -r report.html
   ```

   `-j` activa el ajax spider (Juice Shop es una SPA en Angular y el spider clásico apenas ve rutas). Cronometrarlo: si pasa de 30 minutos, limitar el spider con `-m` o dejar el full scan solo para `main` y `zap-baseline.py` para pull requests.
2. Revisar qué alertas `High` salen. Si hay alguna, G5 se demostrará bloqueando; si no, G5 se queda en «≥ 1 High» igualmente y se explica en el informe que no saltó (no se baja el umbral para forzarlo).
3. Redibujar en draw.io el diagrama 1.b del documento de planteamiento.
4. Uso responsable: ZAP solo contra `localhost` o contra nuestro despliegue, nunca contra la demo pública de Juice Shop.

Fase 2 · Motor de quality gates (antes del 24 nov)

Un solo script para los seis gates, así el umbral y el formato del resumen son iguales en todos. C escribe la estructura y el parser de ZAP; B añade los de sus cuatro herramientas en el diccionario `PARSERS`.

```
import argparse
import csv
import json
import os
import sys

ZAP_RISK = ["Informational", "Low", "Medium", "High"]


def zap_ignored(rules_file):
    if not rules_file or not os.path.exists(rules_file):
        return set()
    with open(rules_file) as f:
        return {row[0] for row in csv.reader(f, delimiter="\t") if len(row) >
1 and row[1] == "IGNORE"}


def zap(report, threshold, ignore):
```

```
    with open(report) as f:
        data = json.load(f)
    minimum = ZAP_RISK.index(threshold)
    skip = zap_ignored(ignore)
    rows = []
    for site in data.get("site", []):
        for alert in site.get("alerts", []):
            if int(alert["riskcode"]) >= minimum and alert["pluginid"] not in
skip:
                rows.append((ZAP_RISK[int(alert["riskcode"])], alert["name"],
f"{alert.get('count', '?')} instancias"))
    return rows


PARSERS = {"zap": zap}


def summary(tool, threshold, rows):
    path = os.environ.get("GITHUB_STEP_SUMMARY")
    if not path:
        return
    with open(path, "a") as f:
        f.write(f"### Quality gate {tool} (umbral {threshold})\n\n")
        f.write(f"Hallazgos que superan el umbral: **{len(rows)}**\n\n")
        if rows:
            f.write("| Severidad | Hallazgo | Detalle |\n| --- | --- | ---
|\n")
            for sev, name, detail in rows[:50]:
                f.write(f"| {sev} | {name} | {detail} |\n")


def main():
    p = argparse.ArgumentParser()
    p.add_argument("--tool", required=True, choices=PARSERS)
    p.add_argument("--report", required=True)
    p.add_argument("--threshold", required=True)
    p.add_argument("--mode", choices=["warn", "block"], default="block")
    p.add_argument("--ignore")
    a = p.parse_args()

    rows = PARSERS[a.tool](a.report, a.threshold, a.ignore)
    summary(a.tool, a.threshold, rows)
    if not rows:
        print(f"Quality gate {a.tool}: superado")
        return
    level = "warning" if a.mode == "warn" else "error"
```

```
    print(f"::{level} title=Quality gate {a.tool}::{len(rows)} hallazgos
superan el umbral {a.threshold}")
    sys.exit(1)


if __name__ == "__main__":
    main()
```

Los falsos positivos de ZAP se gestionan con `security/zap/rules.tsv` (`<pluginid>\tIGNORE\t<motivo>`): el script descarta esos plugins al contar, así el gate no depende de cómo trate cada versión de ZAP la regla en el informe.

Fase 3 · Jobs `dast`, `security-gate` y `notify` (18–24 nov)

1. **`dast`** (`needs: build`): descargar `image.tar`, `docker load`, `docker run -d -p 3000:3000`, esperar a que responda (bucle con `curl` y `timeout 180`), `zaproxy/action-full-scan` con `cmd_options: -a -j -x report_xml.xml`, `fail_action: false` y `allow_issue_writing: false`. Después: subida con `dd-upload.sh "ZAP Scan" report_xml.xml "DAST - ZAP"` (`if: always()`), artefacto `report-dast` y G5 **sin** `continue-on-error`:

   ```
   python3 scripts/quality_gate.py --tool zap --report report_json.json --
   threshold High --ignore security/zap/rules.tsv --mode block
   ```

2. **`security-gate`** (`needs: [sca, sast, iac, container, dast]`, `if: always()`): descargar todos los `report-*` y ejecutar G6, bloqueante: Dependency-Check con umbral 9.0 y Trivy con `CRITICAL`. Este job es el *required status check* de `main`.
3. **`notify`** (`needs: [dast, security-gate, deploy]`, `if: failure()`): enviar el enlace al run por Discord si existe el secreto.

   ```
   curl -sS -H "Content-Type: application/json" \
     -d "{\"content\":\"Pipeline bloqueado por un quality gate: $RUN_URL\"}"
   "$DISCORD_WEBHOOK"
   ```

   con `RUN_URL: ${{ github.server_url }}/${{ github.repository }}/actions/runs/${{ github.run_id }}` en `env`.
4. Lanzar un push a `main` y comprobar el comportamiento esperado: build en verde con avisos, `security-gate` en rojo, `deploy` saltado, aviso recibido. **Guardar este run: es la evidencia obligatoria.**

Fase 4 · Triage en DefectDojo (24–28 nov)

1. Configurar SLA por severidad y notificaciones del producto.

2. Revisar los hallazgos Critical y High de las cinco herramientas: marcar *Verified* los reales y asignarlos.
3. Marcar al menos un **falso positivo** justificado (típico: una alerta de ZAP sobre una ruta del frontend, o un secreto de Semgrep en datos de prueba) y reflejarlo en el fichero de supresión de la herramienta.
4. Registrar al menos una **aceptación de riesgo** con motivo y fecha de expiración (los CVE de la rama `demo/gate`).
5. Comprobar la **deduplicación**: lanzar dos runs seguidos y ver que los tests no crecen; localizar un hallazgo marcado como duplicado entre herramientas.
6. Con B, ejecutar la rama `demo/gate` y comprobar que pasa G6 y llega a `deploy`.

Fase 5 · Evidencias, informe y entrega (25 nov – 2 dic)

1. Reunir `evidencias/`: logs descargados de los runs (fallido y `demo/gate`), informes de cada herramienta, capturas de DefectDojo (engagement, hallazgos por severidad, duplicado, falso positivo, riesgo aceptado, métricas).
2. Escribir los apartados 3 (diagrama 1.b), 4 (G5 y G6), 5 (resultados de ZAP), 6 (gestión en DefectDojo) y 7 (conclusiones y respuesta al cliente).
3. Maquetar el informe completo, controlar que no pasa de 15 páginas sin portada ni anexos y que los anexos solo contienen evidencias.
4. Montar `PAI4-ST20.zip` (informe, `codigo/`, `evidencias/`), revisar que no hay credenciales y subirlo a la Enseñanza Virtual antes del 2 dic a las 23:59.

Checklist Rol C:

- [ ] ZAP probado en local y tiempo de escaneo controlado
- [ ] Diagrama 1.b en draw.io
- [ ] `quality_gate.py` con estructura y parser de ZAP
- [ ] Jobs `dast` (G5 bloqueante), `security-gate` (G6) y `notify`
- [ ] Run de `main` fallando por gate guardado como evidencia
- [ ] Triage: verificados, 1 falso positivo, 1 riesgo aceptado, duplicado comprobado
- [ ] Run de `demo/gate` pasando y desplegando
- [ ] Apartados 3 (1.b), 4 (G5–G6), 5 (ZAP), 6, 7 y anexos
- [ ] Zip montado, revisado y entregado

Coordinación y calendario

Cada seguimiento tiene un objetivo de equipo concreto; si algo de A se retrasa, B y C siguen trabajando en local y no se bloquean.

Hito	Rol A	Rol B	Rol C
Seguimiento 1 (17/18 nov): diseño + plataforma	Repo, DefectDojo con túnel, secretos, `dd-upload.sh` probado, `build` funcionando	4 herramientas probadas en local, NVD key entregada	ZAP probado en local, diagrama 1.b v1
Seguimiento 2 (24/25 nov): pipeline completo	`k8s/`, `deploy` en `kind`, Dependabot, protección de `main`	Jobs `sca`, `sast`, `iac`, `container` subiendo a DefectDojo; parsers del gate	`quality_gate.py`, `dast`, `security-gate`, `notify`; run fallido por G6 guardado
1 dic: cierre	Diagrama 1.a final, apartados 1 y 3	Rama `demo/gate` con C, apartados 2, 4 y 5	Triage, evidencias, apartados 3, 4, 5, 6, 7, maquetación
2 dic 23:59: entrega	Revisa el informe	Revisa el informe	Monta y sube `PAI4-ST20.zip`

Tareas comunes:

- [ ] Reunión corta tras cada seguimiento para aplicar lo que diga el profesor y actualizar los diagramas si cambia el diseño
- [ ] Cada PR a `main` lo revisa alguien de otro rol
- [ ] Revisión cruzada del informe completo antes del 2 dic: diagramas = YAML, todas las herramientas con hallazgo demostrado, al menos un gate bloqueando de verdad
- [ ] Comprobación final de que no hay credenciales ni en el repo ni en el zip
- [ ] Uso de IA declarado en el informe de forma ética y responsable, como pide la presentación